

RAG

Sep 30, 2026 · @Salike Hassan

RAG (Retrieval-Augmented Generation) = Retrieve the relevant chunks from your data, Augment the prompt with them, then Generate an answer grounded in that context, instead of sending the whole document to the LLM.

Note key. *My way* = my own explanation. *Correction* = something I said that needs fixing. *Addition* = something missing from my explanation. *Analogy / Insight* = my own connections. *CCA tip* = exam-relevant point. *Tech note* = plain technical detail.

1. What RAG is and why we need it

RAG = Retrieve, Augment, Generate. It grounds the LLM's answer in your own data by retrieving only the relevant pieces and inserting them into the prompt before generation.

Why we need it. Whichever LLM we use, it has two limits:

- **Context window:** only so much text fits in one prompt.
- **Cost:** pricing is per token, so sending everything is expensive.

My way (example). Take a very large document from a healthcare product company. It holds information from the software engineering department, the HR department and the medical department. A user wants one specific answer out of it. We cannot send the whole document to the LLM and expect it to find the answer: it will not fit in the context window, and it would cost too much. RAG solves this by sending only the small, relevant parts.

Analogy. A human brain cannot hold a whole 50-minute lecture in working memory, so we take notes and refer back to them. RAG applies the same pattern to the model: retrieve just the relevant notes and hand them over at the moment they are needed.

RAG also avoids the alternative of fine-tuning a new model every time the data changes.

2. The 7-step pipeline

Retrieval covers steps 1 to 5, augmentation is step 6 and generation is step 7.

Step	What happens
1. Chunk	Split source documents into manageable pieces
2. Embed	Convert each chunk into a vector embedding
3. Store	Save embeddings in a vector database / search index
4. Query	Embed the user's question the same way
5. Retrieve	Find the chunks whose embeddings are closest to the query embedding
6. Augment	Insert the retrieved chunks into the prompt as context
7. Generate	The LLM generates an answer grounded in the retrieved context

Analogy. This maps onto Azure AI Search patterns from my Document Intelligence work: index the documents, run a similarity / vector search, then hand the top results to the model as grounding context instead of letting it answer from memory alone.

Full flow (corrected and complete). User asks a question, the question is embedded into a vector, the vector database is searched for similar chunks (semantic search), a BM25 lexical search runs on the same query, both result sets are merged and ranked, the top chunks are injected into the prompt (augmentation), the enriched prompt goes to the LLM, and the LLM answers from the actual document content.

3. Chunking (step 1)

Chunking means retrieving information in small pieces instead of the whole document. The strategy depends on the use case.

My way. Three main strategies: **size-based** (with overlap so the text stays complete across boundaries), **structure-based** (when the document has clear sections) and **semantic** (more complex, but tends to give better results).

Strategy	How it works	Trade-off
Fixed-size (size-based)	Split every N tokens/characters, often with overlap	Simple and fast, but can cut a sentence or idea in half
Recursive character	Try splitting on paragraph, then sentence, then word boundaries in order	Respects natural structure better than pure fixed-size
Semantic chunking	Split where embedding similarity between adjacent sentences drops	Keeps topics together, but costlier to compute
Sliding window / overlap	Each chunk shares some tokens with its neighbour	Reduces "context lost at the boundary", increases storage
Document-structure-aware	Split by headings, sections, table boundaries	Best fidelity for structured docs (contracts, manuals), needs parsing logic

Addition. Recursive character splitting was not in my explanation. Overlap is a technique used with size-based chunking, not a separate strategy.

Tech note. Chunk size is a real trade-off. Too small and a chunk loses context (a sentence about "the fee" with no mention of which fee). Too large and irrelevant text dilutes the embedding, so retrieval is less precise, and it wastes context-window budget once inserted into the prompt.

4. Embeddings and vector search (steps 2 to 5)

After chunking, each chunk is converted into a text embedding and stored in a vector database, so we can run a vector search on top of it.

My way. When the user asks a question, we create an embedding of that query, go to the vector database and search for the stored chunks that are closest to it.

Correction. I said embeddings convert text into a score from -1 to 1. That is not quite right. An **embedding is a vector**: a list of hundreds or thousands of numbers representing the meaning of the text in mathematical space. The -1 to 1 range is the **cosine similarity score** you get when you *compare two embeddings*. Important distinction for the CCA exam.

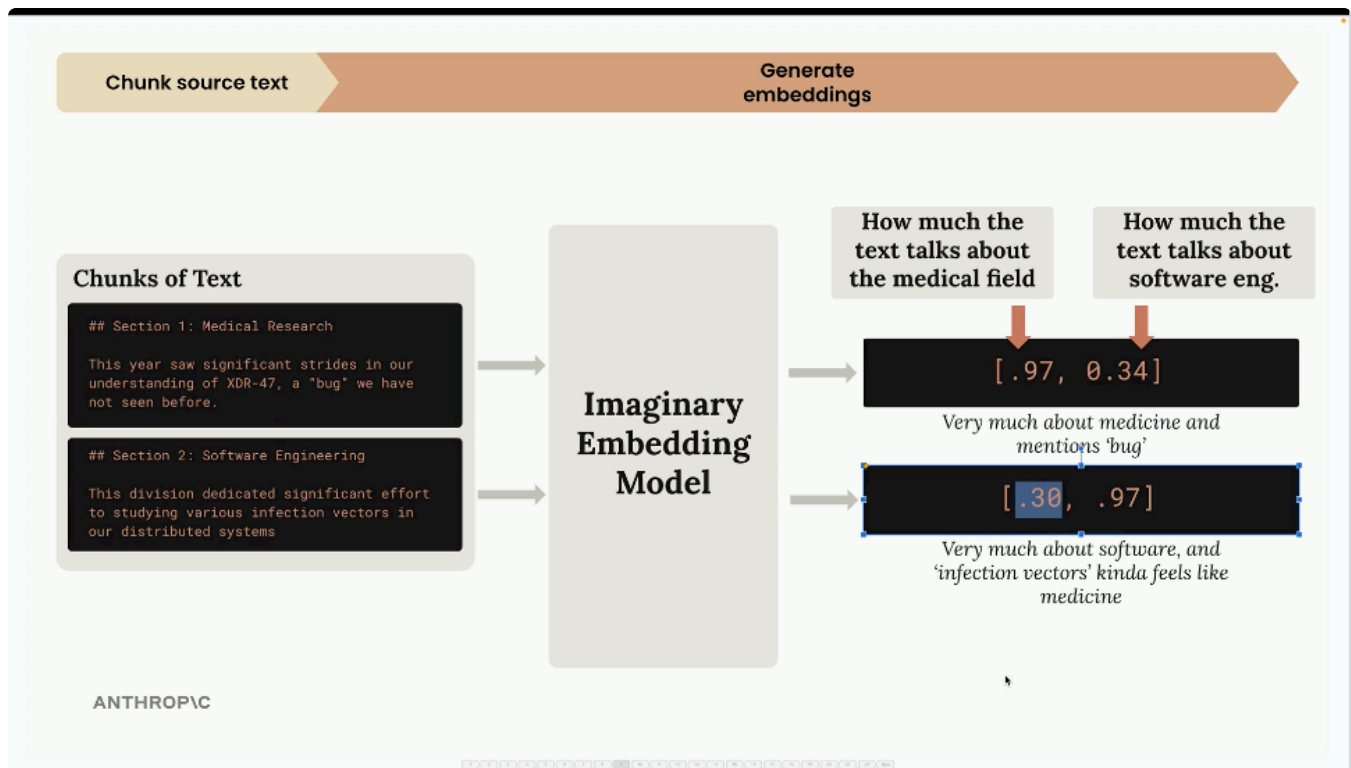
Term	What it is
Embedding	A vector (list of numbers) representing the meaning of a text
Vector database	Stores the embeddings and supports similarity search
Cosine similarity	Score from -1 to 1 comparing two embeddings; closer to 1 means more similar

The query must be embedded the same way as the chunks (step 4 of the pipeline), otherwise the comparison is meaningless.

4.1 Worked example (course slides)

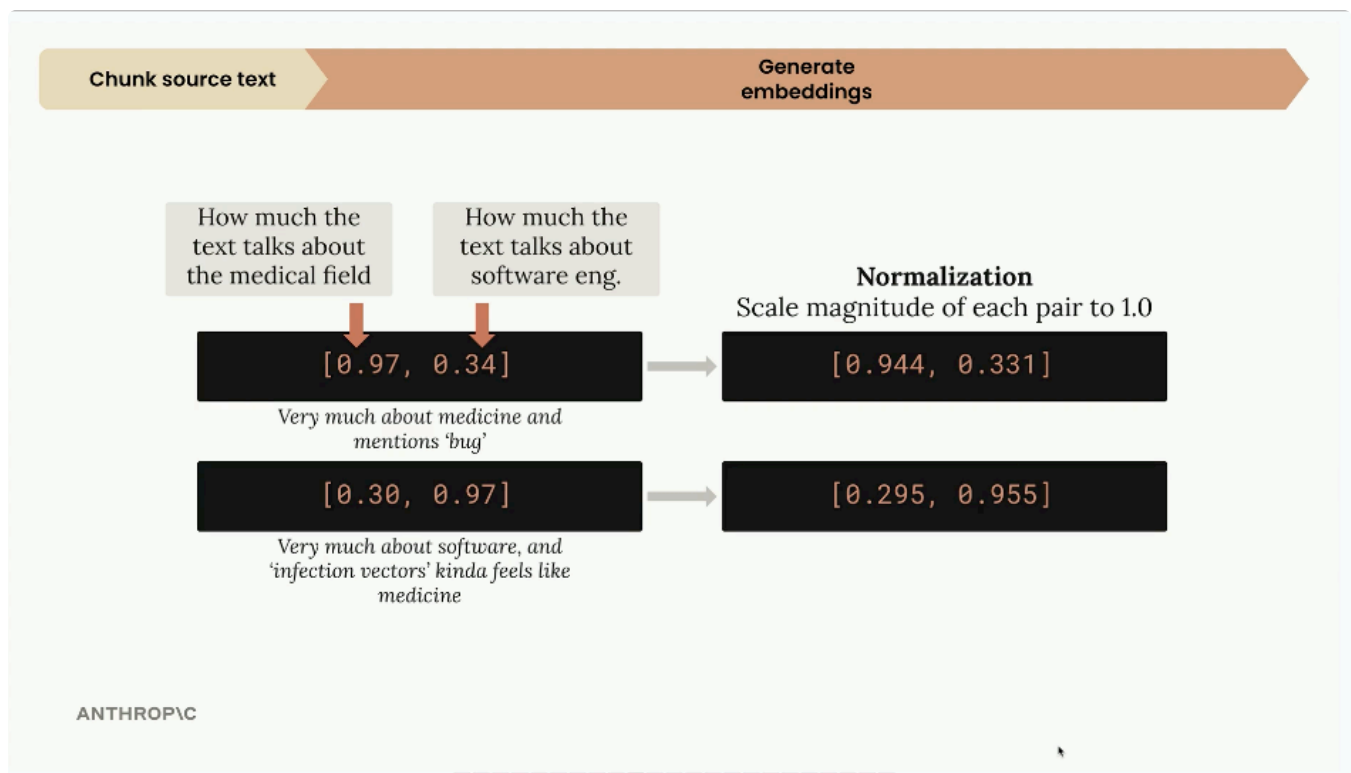
A toy model with only two dimensions shows how embeddings and cosine similarity work end to end. The two chunks come from the healthcare-style document: Section 1 is Medical Research, Section 2 is Software Engineering.

Step 1: generate embeddings. The imaginary model scores each chunk on two dimensions: how much it talks about the medical field and how much it talks about software engineering.



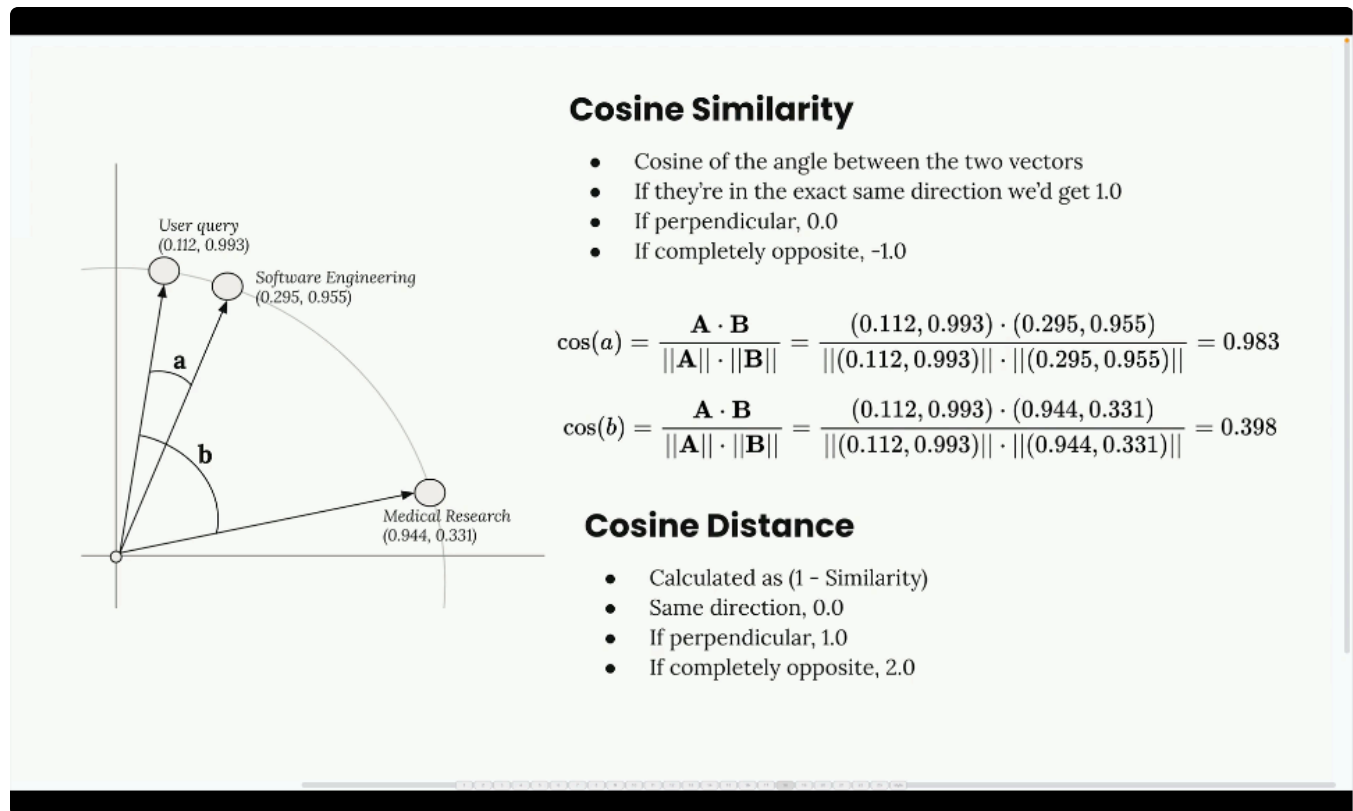
The medical chunk is very much about medicine and mentions "bug". The software chunk is very much about software, and "infection vectors" kind of feels like medicine, which is why it still scores 0.30 on the medical dimension.

Step 2: normalization. Each vector is scaled so its magnitude is 1.0.



Tech note. Once vectors have magnitude 1.0, cosine similarity is just the dot product of the two vectors, because the denominator (the two magnitudes multiplied) becomes 1. That is why normalizing makes similarity search cheap.

Step 3: compare the query with each chunk. The user query embeds to (0.112, 0.993), which points mostly toward the software dimension.



Cosine similarity	Meaning	Cosine distance (1 - similarity)
1.0	Exact same direction	0.0
0.0	Perpendicular (unrelated)	1.0
-1.0	Completely opposite	2.0

So the query is far closer to the Software Engineering chunk, and that chunk is the one retrieved. This is the -1 to 1 score from the correction above, and the distance scale is what a vector database such as Redis returns (see section 10).

Correction. Recalculating with the rounded numbers on the slide, the query against the Medical Research vector comes to about 0.43, not the 0.398 shown, and the Software Engineering one to about 0.98. The conclusion is the same, but do not memorise 0.398 as an exact figure.

5. Retrieval techniques: semantic, BM25 and hybrid

The vector search alone can return results that are not relevant. To fix that, combine it with a lexical search and merge the results.

My way. The vector search is the **semantic search**. Alongside it we run a **lexical search**: convert the query into tokens and look for those tokens in the documents. The token that is least repeated gets the highest priority. Then merge the semantic results and the lexical results, and the chunks with the highest ranking are the information we are looking for.

(Best Match 25). It scores documents by how often a search term appears and gives **higher weight to rare terms**, which matches my explanation.

Technique	How it works	Best for
Semantic / dense retrieval	Compare query embedding vs chunk embeddings (cosine similarity)	Conceptual or paraphrased queries, meaning-based matching
Keyword / sparse retrieval (BM25)	Term-frequency matching, no embeddings involved	Exact terms, IDs, codes, names that embeddings can blur
Hybrid search	Combine dense + sparse scores (weighted or reciprocal-rank-fused)	Production systems: semantic recall plus exact-match precision
Reranking	A second, more expensive model re-scores the top-K candidates	Squeezing more precision out of the final short list before the prompt

Technique	How it works	Best for
Metadata filtering	Pre-filter by structured fields (date, department, doc type)	Narrowing scope in large multi-tenant or multi-source knowledge bases

Analogy. Hybrid search is like searching my own email. Outlook's keyword search finds an exact invoice number instantly, but if I only vaguely remember "that email about the server outage last month", keyword search fails and I need something that understands meaning. A production RAG system needs both.

Insight. Reranking is the eval-workflow pattern applied inside retrieval: cheap, broad retrieval gets a wide candidate set fast, then a more expensive, more accurate pass re-orders just that shortlist. Same "cheap-first, expensive-second" shape as a fast filter before a model-based grader.

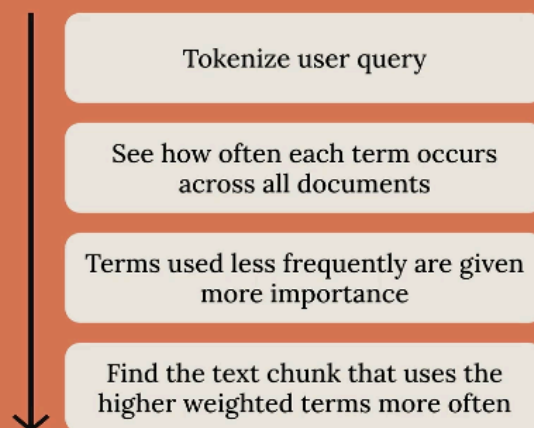
5.1 Hybrid search in detail: lexical (BM25) + semantic

Lexical search (BM25). Many methods exist for text search, but BM25 is the one commonly used in RAG pipelines. BM25 = Best Match 25; the 25 means it was the 25th variation of a formula by its original creators.

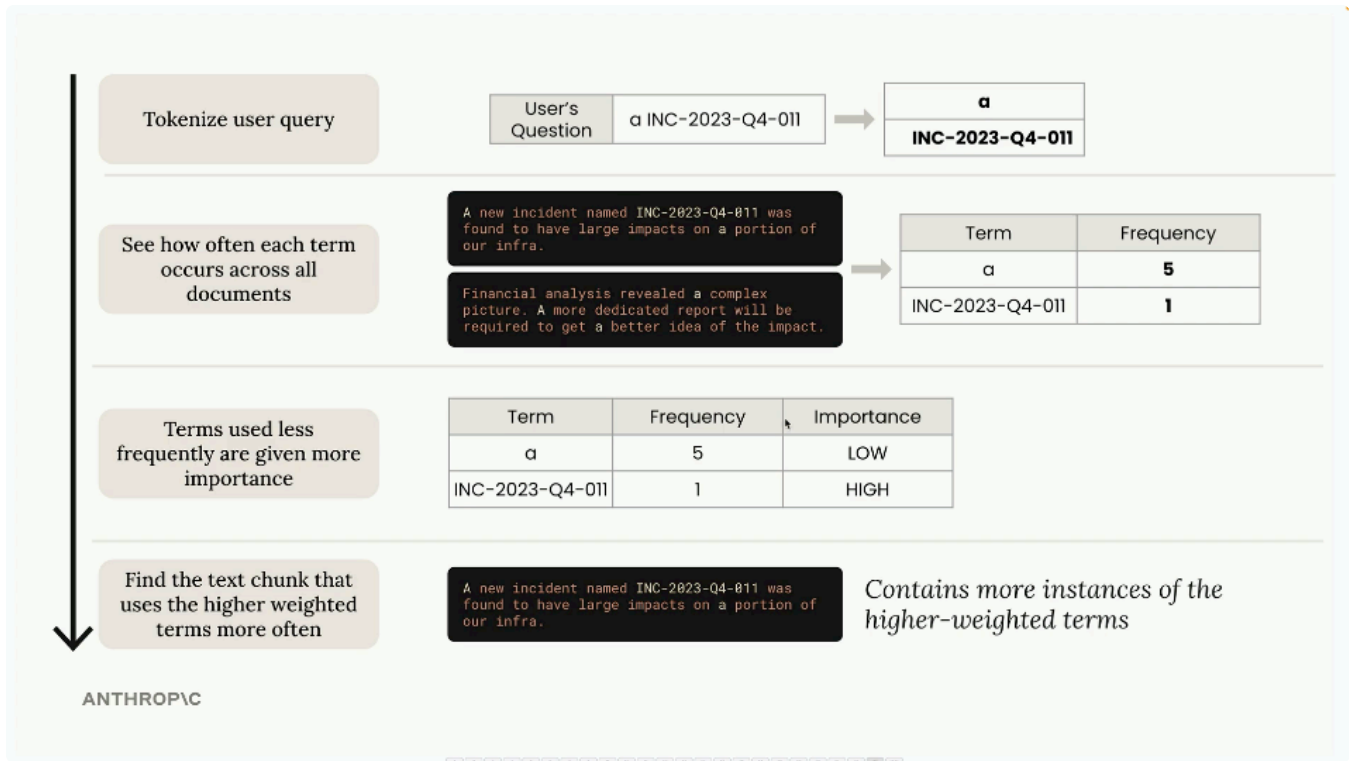
Lexical Search: BM25

- **Many** methods for implementing text search, but BM25 is commonly used in RAG pipelines.
- **BM25 = Best Match 25.** The '25' means this was the 25th variation of a formula developed by the original creators.

ANTHROPIC

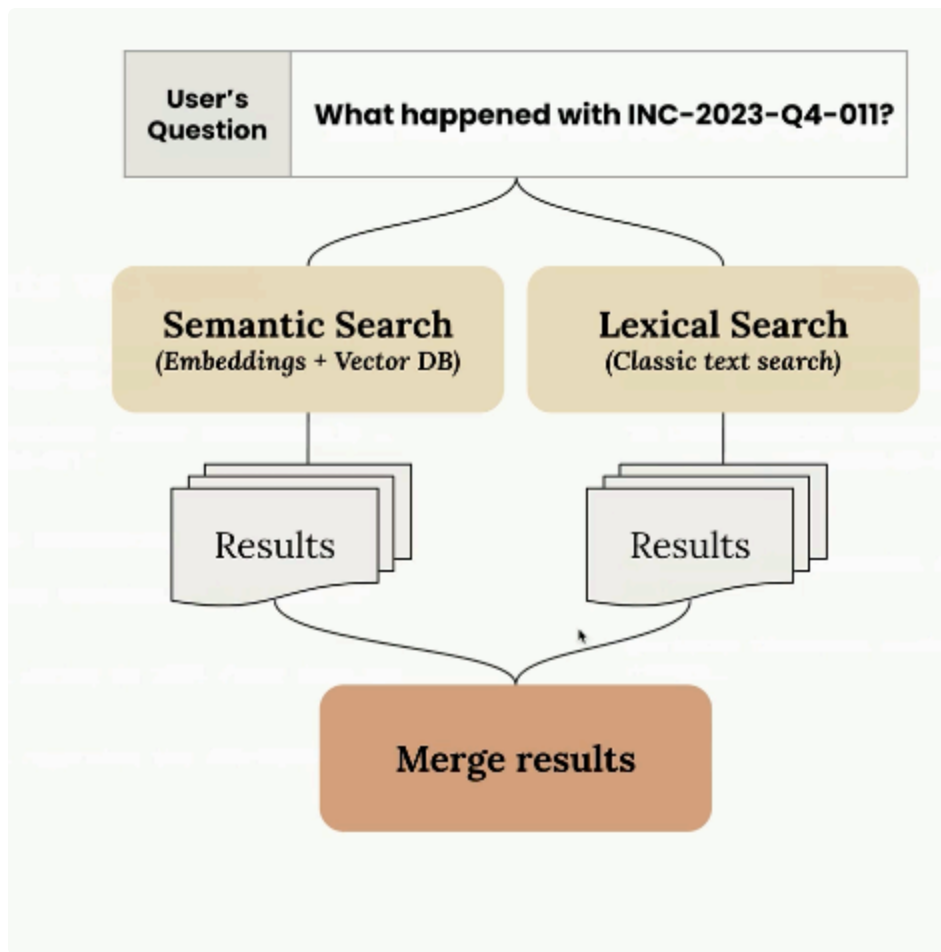


Example. The question is "a INC-2023-Q4-011". After tokenizing, the common word "a" occurs 5 times across the documents (low importance) and the incident ID occurs once (high importance). The chunk that mentions the incident ID wins.

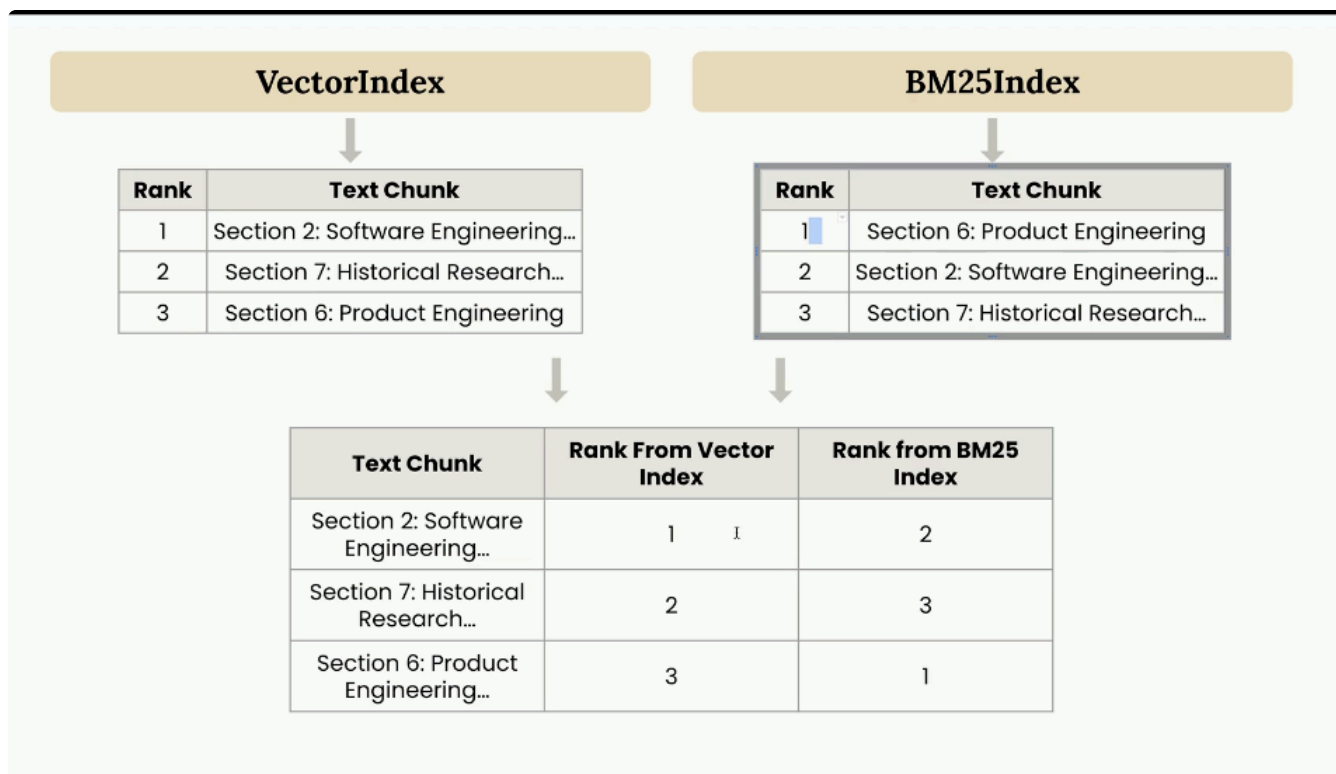


Tech note. This is exactly why hybrid search exists. An exact ID like INC-2023-Q4-011 is easy for BM25 to find but easy for embeddings to blur, while a vague, paraphrased question is easy for embeddings and hard for BM25. Running both covers both cases.

Two searches, one merge. The same user question goes to the semantic search (embeddings + vector DB) and the lexical search (classic text search). Each returns its own ranked results, and the results are merged.



Merging by rank. The two searches return different orders, and their scores are not on the same scale, so the merge uses each chunk's **rank** from each index rather than its raw score.



Reciprocal Rank Fusion (RRF). Each chunk gets a score of $1 / (k + \text{rank})$ from every index, and the scores are added. A higher score means a more relevant chunk. In the slide k is set to 1 for clearer numbers; 60 is often used in practice.

$$\text{RRF_score}(d) = \sum_{i=1}^n \frac{1}{k + \text{rank}_i(d)}$$

k is a constant, 60 is often used, but going to use 1 here for clearer results

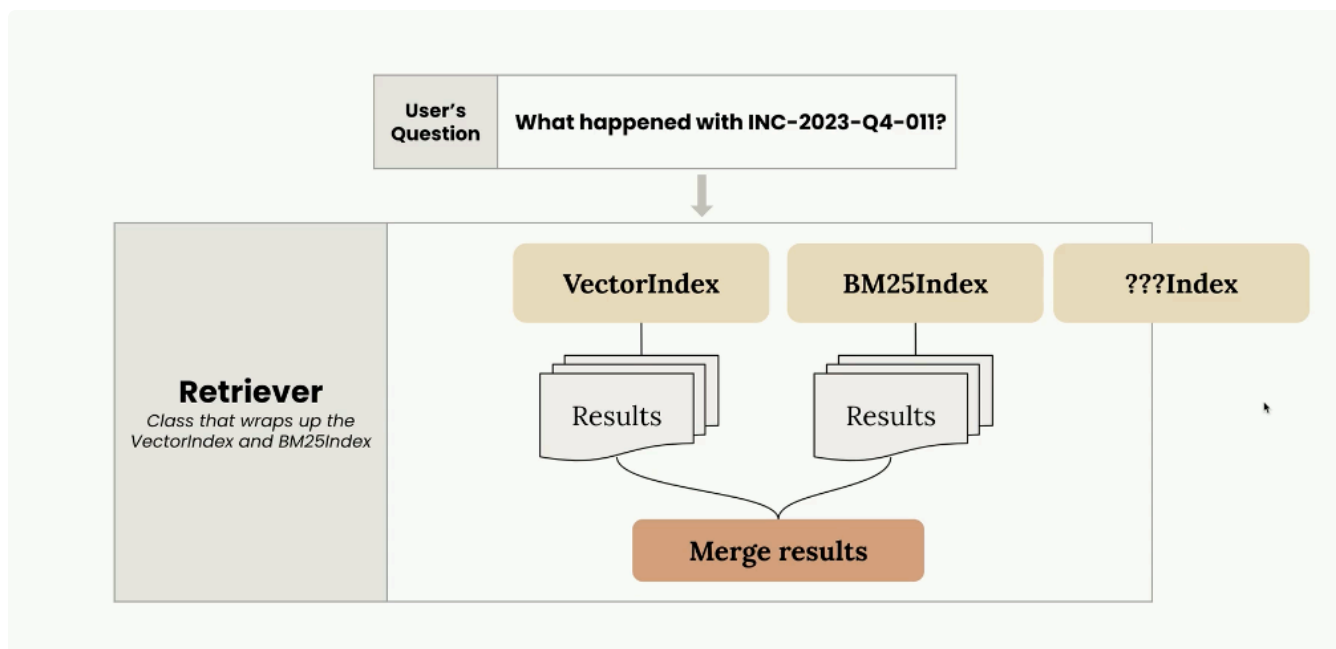
Text Chunk	Rank From Vector	Rank from BM25	Score	Final Rank
Section 2: Software Engineering...	1	2	$1.0 / (1 + 1) + 1.0 / (1 + 2) = \mathbf{0.833}$	1
Section 6: Product Engineering...	3	1	$1.0 / (1 + 3) + 1.0 / (1 + 1) = \mathbf{0.75}$	2
Section 7: Historical Research...	2	3	$1.0 / (1 + 2) + 1.0 / (1 + 3) = \mathbf{0.583}$	3

Higher score = more relevant text chunk

Chunk	Rank from vector	Rank from BM25	RRF score (k = 1)	Final rank
Section 2: Software Engineering	1	2	$1/(1+1) + 1/(1+2) = 0.833$	1
Section 6: Product Engineering	3	1	$1/(1+3) + 1/(1+1) = 0.75$	2
Section 7: Historical Research	2	3	$1/(1+2) + 1/(1+3) = 0.583$	3

Insight. Product Engineering was last in the vector results but first in BM25, and it still ends up second overall. A chunk that ranks well in both lists beats a chunk that ranks first in only one, which is the whole point of merging.

Retriever class. In code, a Retriever class wraps the VectorIndex and BM25Index and does the merge, so the rest of the app just calls one retriever. The "???"Index" box on the slide shows it can be extended with more indexes later.



Addition. This RRF merge is the same fusion step Azure AI Search uses for hybrid queries in section 9.

6. Augmentation and generation (steps 6 and 7)

My way. The extracted information is sent to the LLM, and we get the answer out of it. I covered generation only briefly at the end.

Addition. I skipped the augmentation step. **Augmentation** is where the top-ranked chunks from the combined semantic + BM25 search are **injected into the prompt**. Instead of sending the raw user question, we send: "Here is the relevant context from our document: [retrieved chunks]. Now answer this question: [user question]." That injection IS the augmentation step.

Generation is the LLM writing its answer from that enriched prompt. The LLM is not working from memory; it works from your actual documents in real time, and that is what makes RAG powerful.

7. The agentic RAG loop

Basic RAG is single-pass: embed once, retrieve once, generate once. Agentic RAG wraps a decision loop around the pipeline so Claude can judge whether what it retrieved is sufficient, and act if it is not.

Step	What Claude decides
1. Query planning	Break a complex question into sub-questions before retrieving
2. Initial retrieval	Run retrieval for the first sub-question (or the whole query)
3. Self-critique	Do these retrieved chunks actually answer the question?
4. Query reformulation	If not sufficient, rewrite the query (different terms, narrower or broader scope) and retrieve again
5. Multi-hop retrieval	Use a fact found in one round to inform the next round's query
6. Source selection	Decide which knowledge source or index to query when there are several

Step	What Claude decides
7. Stopping condition	Decide it has enough grounding to answer, or has exhausted reasonable attempts

Insight. This is the Workflow-vs-Agent distinction applied to search. Basic RAG is a **workflow**: retrieve-then-generate, fixed, code controls the flow. Agentic RAG is an **agent** pattern: Claude decides how many retrieval rounds it needs, when to reformulate, and when it has enough. Same retrieval mechanics, different control model around them.

8. Common failure modes

- **Lost in the middle:** with many chunks in a long context, the model attends less reliably to information buried in the middle than at the start or end.
- **Chunk-boundary loss:** the key fact is split across two chunks and neither alone contains enough to retrieve correctly.
- **Retrieval-generation mismatch:** the right chunks were retrieved, but the prompt does not make clear the model should prioritise them over its training knowledge.
- **Over-retrieval:** pulling in too many chunks "just in case" burns context budget and brings back lost-in-the-middle.
- **Stale index:** the vector store was not updated after the source documents changed, so retrieval confidently returns outdated grounding.

CCA tip. The exam distinguishes **retrieval failure** (wrong or no chunks found) from **grounding failure** (right chunks retrieved, but the model still did not use them correctly). They are fixed differently: retrieval failures by better chunking, retrieval or reranking; grounding failures by better prompting and instructions.

9. RAG in Azure AI Search

Azure AI Search is the retrieval half of RAG on Azure: one index holds the chunks, their vectors and their metadata, one query can run keyword and vector search together, and the top chunks go to the LLM (Azure OpenAI) for augmentation and generation.

9.1 Pipeline step to Azure AI Search feature

RAG step	Azure AI Search feature
1. Chunk	Built-in chunking or the Text Split skill in a skillset, so each chunk becomes its own search document
2. Embed	Integrated vectorization with an embedding model (Azure OpenAI, or a custom one), at indexing time and at query time
3. Store	An index with a vector field (<code>Collection(Edm.Single)</code> , with dimensions) beside text and filterable metadata fields
4. Query	The query text is vectorized with the same model (a vectorizer on the field, or your own code)
5. Retrieve	Hybrid query: BM25 keyword search plus vector search, merged into one ranked list; optional semantic ranker on top
6. Augment	Your app puts the top-k chunks into the prompt
7. Generate	Azure OpenAI generates the answer from those chunks

Addition. In Azure AI Search the keyword side of hybrid search is the BM25 ranking from section 5, the merge of keyword and vector results uses Reciprocal Rank Fusion (RRF), and the **semantic ranker** is the built-in reranking step from the same section.

9.2 Classic RAG query flow

1. The user asks a question; the app embeds it with the same model used at indexing.
2. The app sends **one** request to the index containing the query text, the query vector and a security filter.
3. Azure AI Search runs keyword and vector search in parallel, merges with RRF and optionally re-scores with the semantic ranker.
4. The app takes the top chunks (top-k) and injects them into the prompt.
5. The LLM answers from those chunks and the app returns the answer with the source references.

The newer alternative is **agentic retrieval**: an LLM plans the query, splits it into sub-queries, runs them in parallel across knowledge sources and returns results with citations. It is the service-side version of the agentic RAG loop in section 7.

9.3 Large embedding model

Tech note. I am assuming the large text embedding model is Azure OpenAI `text-embedding-3-large`, which produces 3,072-dimension vectors by default and can return fewer dimensions if requested. Three rules apply whichever model is used:

- The vector field's `dimensions` must match the model's output size.
- Chunks and queries must be embedded with the **same** model and dimensions.
- Changing the model or dimensions means re-embedding and re-indexing everything.

Larger vectors usually retrieve better but cost more storage and query time, which is the trade-off to weigh against a smaller model.

9.4 Security trimming

Security trimming means each user only gets chunks they are allowed to see. It must happen **inside retrieval**, so restricted text never reaches the prompt; filtering the answer afterwards is too late. Azure AI Search offers two approaches.

Approach 1: security filter (documented and GA). Store the allowed identities on every chunk, then filter at query time.

1. Add a filterable collection field to the index (`retrievable: false` keeps it out of responses, but it is not a security control by itself):

```
{ "name": "group_ids", "type": "Collection(Edm.String)", "filterable": true,
  "retrievable": false }
```

2. At indexing time, copy the allowed group or user IDs from the source into `group_ids` on every chunk of a document.
3. At query time, look up the signed-in user's group IDs (typically from Microsoft Entra ID) and pass them as a filter:

```
{ "filter": "group_ids/any(g:search.in(g, 'group_id1, group_id2'))" }
```

`search.in()` is designed for this case and stays fast even with long lists of IDs. The same filter applies to the keyword and the vector part of a hybrid query.

Approach 2: built-in access control list (ACL) support. Microsoft's documentation points to this as the preferred option when your solution can use it, because the index carries document-level permissions instead of your own filter field. Check the current Microsoft Learn page for its status and limits before choosing it.

Insight. In the healthcare example from section 1, security trimming is what stops an HR user's question from retrieving medical-department chunks: the metadata filter from section 5 is enforced as a hard permission rule, not just a relevance narrowing.

Tech note. Trimming is only as good as the ACL data on the chunks. If a source document's permissions change, its chunks must be re-indexed, otherwise the index serves a stale access list. This is the security version of the "stale index" failure mode in section 8.

Sources: [Security trimming in Azure AI Search](#) · [RAG and Azure AI Search overview](#)

10. Semantic caching for latency (CP America)

A semantic cache answers a repeated or near-repeated question from Redis instead of running the full RAG pipeline again, which removes the retrieval and LLM latency (and cost) for those questions.

My way. For CP America, to deal with latency, I implemented an embedded Redis cache that uses a semantic score to decide whether a cached answer can be reused.

10.1 How it works

1. Embed the incoming question (one embedding call, using the same model as the rest of the pipeline).
2. Vector-search the cache in Redis for the most similar previously answered question.
3. **Cache hit:** if the similarity score is at or above the threshold, return the stored answer immediately. Skip Azure AI Search and the LLM.
4. **Cache miss:** run the normal RAG flow (sections 2 and 9), then store the question embedding and the answer in Redis for next time.

An ordinary cache only matches identical text. A semantic cache matches on meaning, so "How do I reset my password?" and "password reset steps?" can share one cached answer. This is the same embedding and cosine-similarity idea from section 4, applied to questions instead of document chunks.

10.2 Things to get right

Concern	Why it matters
Similarity threshold	Too low returns a cached answer to a different question (wrong answer); too high and the cache rarely hits. Tune it on real questions.
Score direction	Redis vector search commonly returns a distance, where smaller is closer; cosine distance is 1 minus cosine similarity. Compare against the threshold in the right direction.
Security trimming	A cached answer was built from chunks one user was allowed to see. Cache entries must be scoped by the user's permission group (or the cache skipped for restricted content), or one user can be served another's restricted answer.
Freshness	Set a TTL and clear entries when the source documents change, otherwise the cache repeats the "stale index" failure mode from section 8.
What is saved	Cache the final answer for the fastest hit, or only the retrieved chunks if answers must be regenerated per user.

Insight. The cache is a cheap-first, expensive-second shape again, like reranking in section 5: a fast similarity lookup in front, and the full retrieve-and-generate pipeline only when the lookup fails.

Tech note. The details above are the general semantic-caching pattern. Send me your actual threshold, TTL and cache-key design and I will replace the general points with what you built.

11. One-liner and quick revision

One-liner. RAG solves the context window and cost problem by retrieving only the most relevant chunks from large documents using vector similarity search and BM25 lexical search, injecting those chunks into the prompt as context, and letting the LLM generate a precise answer from that relevant information rather than the entire document.

Term	Remember
RAG	Retrieve, Augment, Generate
Why	Context window limit + cost
Chunking	Size-based (with overlap), structure-based, semantic
Embedding	A vector; cosine similarity (-1 to 1) compares two of them
BM25	Best Match 25, lexical search, rare terms weigh more (not "PM24")
Hybrid search	Semantic + BM25, results merged and ranked
Reranking	Second, costlier pass over the top-K candidates
Augmentation	Injecting retrieved chunks into the prompt
Agentic RAG	Claude loops: plan, retrieve, self-critique, reformulate, stop
Retrieval vs grounding failure	Wrong chunks vs right chunks used badly